

The folder used in every example

```
~/work/
|-- notes.txt    3 lines of text
|-- report.txt   2 lines of text
|-- todo.txt     4 lines, 3 of them start with buy
|-- photos/      cat.jpg dog.jpg tree.jpg
`-- backup/      empty
```

Make it yourself in two minutes and try every line on this card.

How a command is built

command argument argument what to do, then what to do it to

```
$ cp notes.txt backup/
| | |
| | | '- where to put it
| | | '- ----- what to do it to
| | | '- ----- the command: what to do
```

command option argument an option changes how it is done

```
$ ls -l photos
| | |
| | | '- what to do it to
| | | '- ---- an option: how to do it
| | | '- ----- the command
```

An option starts with - and is case sensitive. -l and -L are different options.

Where am I, and what is here?

pwd print the folder you are in

```
$ pwd
/home/you/work
```

It starts at / and leaves nothing out. That is an **absolute path**.

ls list what is in a folder

```
$ ls
backup notes.txt photos report.txt todo.txt
$ ls photos
cat.jpg dog.jpg tree.jpg
```

No folder named? Then it lists the one you are in.

ls -l list with the details

```
$ ls -l notes.txt
-rw-rw-r-- 1 you you 59 Sep 15 10:30 notes.txt
| | |
| | | | '- when it changed
| | | | '- ---- size in bytes
| | | | '- ----- - = file, d = folder
```

you you is your own username, twice.

ls -a list the hidden entries too

```
$ ls -a photos
. .. cat.jpg dog.jpg tree.jpg
```

. is this folder. .. is the folder above it. A hidden name starts with a dot.

cd folder go into a folder

```
$ cd photos
$ pwd
/home/you/work/photos
```

cd .. goes up one. **cd ../..** goes up two. **cd ~** goes home, and so does **cd** on its own.

Two ways to say where

photos **relative:** a folder inside the one I am in

../report.txt **relative:** up one, then that file

../../work **relative:** up two, then down into work

~/work/photos **absolute:** start at my home folder

/home/you/work **absolute:** start at the very top

Relative is shorter, and it still works after the whole folder is moved. Absolute means the same one place from anywhere.

Make, read, copy, move, delete

mkdir name make a folder

```
$ mkdir letters
$ mkdir letters/sent letters/draft
```

It takes as many names as you give it.

touch file make an empty file

```
$ touch list.txt
```

cat file show what is in a file

```
$ cat notes.txt
Meeting on Monday.
Bring the camera.
Ask about the budget.
```

head -n 2 file show only the first 2 lines

```
$ head -n 2 notes.txt
Meeting on Monday.
Bring the camera.
```

tail -n 1 file show only the last line

```
$ tail -n 1 notes.txt
Ask about the budget.
```

Change the number to get more lines. With no -n at all, both show 10. For a file too long for **cat**.

echo "text" print text

```
$ echo "Hello there"
Hello there
```

cp from to copy it. The original stays

```
$ cp notes.txt backup/
$ ls backup
notes.txt
```

mv from to move it, or rename it

```
$ mv notes.txt backup/      moves it
$ mv notes.txt old-notes.txt renames it
```

Both change the name or the place, so both are **mv**.

rm file delete a file. Permanently

```
$ rm list.txt
```

No Trash. No Undo. Run **pwd** and **ls** first, every time.

rmdir folder delete a folder, if it is empty

```
$ rmdir backup
```

Silence means success. **mkdir**, **touch**, **cp**, **mv**, **rm** and **rmdir** print nothing when they work. If you see a message, something went wrong. Read it.

Type less

Tab	finish the name I started. Type 3 letters, press Tab
Tab Tab	nothing completed? Press again to see every match
Up / Down	step through the commands you already ran
history	list them all, with a number on each
clear	clear the screen. Ctrl+L does the same
Ctrl+C	stop a command that is still running

"name with spaces" quote it, or the shell reads two names

```
$ cd My Photos
bash: cd: too many arguments
$ cd "My Photos"
```

Tab does it for you, with a \ before each space.

* match any text here

```
$ ls photos/*.jpg
photos/cat.jpg  photos/dog.jpg  photos/tree.jpg
```

The **shell** expands * before the command runs. Check what it matches before you attach it to **rm**.

Search inside files, and count

grep word file print every line that contains word

```
$ grep buy todo.txt
buy milk
buy bread
buy stamps
```

3 lines came out, so 3 of the 4 jobs are shopping. grep prints the whole line, not the word. It is **case sensitive**: **Buy** would find none of these.

grep word *.txt search several files at once

```
$ grep buy *.txt
todo.txt:buy milk
todo.txt:buy bread
todo.txt:buy stamps
```

With more than one file it puts the filename in front, so you can tell the hits apart.

wc file count lines, words and bytes

```
$ wc notes.txt
3 10 59 notes.txt
| | |
| | '-' bytes
| | '--- words
| '----- lines
```

The third number is bytes, not letters. For plain English text they are the same. A file with accents or emoji has more bytes than letters, and **wc -m** counts the letters.

wc -l file count the lines only

```
$ wc -l *.txt
3 notes.txt
2 report.txt
4 todo.txt
9 total
```

Several files? wc adds a **total** line at the bottom.

Join commands together

a | b give what a printed to b as its input

```
$ grep buy todo.txt | wc -l
3
```

Read it left to right: *find the buy lines, count them*. grep never opened a file for wc here.

a > file write the output into a file. Replaces it

```
$ ls photos > list.txt
$ cat list.txt
cat.jpg
dog.jpg
tree.jpg
```

Nothing appears on screen when you run the first line. The output went into the file.

a >> file add the output to the end. Keeps the rest

```
$ echo "checked by me" >> list.txt
```

>> adds. > replaces. Mixing them up is the usual way to lose a file.

Putting it together

A real job, start to finish: pull the shopping lines out of **todo.txt**, count them, and keep a copy of the notes beside them.

```
$ cd ~/work
$ mkdir shopping
$ grep buy todo.txt > shopping/list.txt
$ cat shopping/list.txt
buy milk
buy bread
buy stamps
$ wc -l shopping/list.txt
3 shopping/list.txt
$ cp notes.txt shopping/
$ ls shopping
list.txt  notes.txt
```

Six commands, one job. Only the ones with something to show printed anything.

When it goes wrong

No such file or directory	the name is wrong. Check the spelling, the capitals and the .txt
Not a directory	that is a file. You cannot cd into a file
Directory not empty	rmdir only removes empty folders
too many arguments	a name has spaces in it. Quote it
command not found	the command name is wrong. LS is not ls
cannot access '*.png'	nothing matched the *
Permission denied	the file is not yours to change. Leave it alone

ls --help ask a command to list its own options

```
$ ls --help
Usage: ls [OPTION]... [FILE]...
  -a, --all      do not ignore entries starting with .
  -l            use a long listing format
```

Long output? Try **ls --help | head -20**.

man ls the full manual for a command

```
$ man ls
```

Arrow keys scroll. Press **q** to quit.

Stuck? Five checks, in order. 1. Where am I? **pwd** 2. What is here? **ls** 3. Is the name exact? Check spelling, capitals, spaces. 4. What does **--help** or **man** say? 5. Try again with the corrected name.

Ubuntu counts capitals. **notes.txt** and **NOTES.txt** are two different names. **ls** and **LS** are two different commands. Type every name the way **ls** shows it.